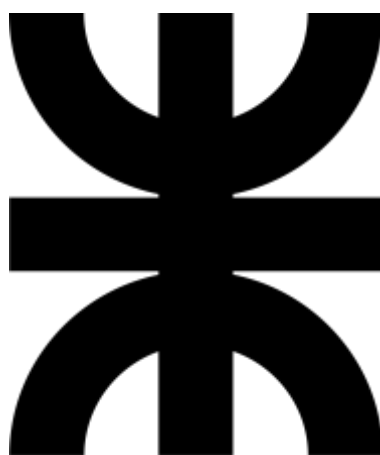


Universidad Tecnológica Nacional

Facultad Regional Córdoba

Ingeniería en Sistemas de Información

Cátedra de Ingeniería y Calidad de Software



Unidad Nro. 4: Aseguramiento de Calidad de Proceso y de Producto

Trabajo Práctico Nro. 6: Test Driven Development (TDD)

Año Lectivo: 2025

Curso: 4K3 | Grupo 6

Docentes:

- Covaro, Laura
- Massano, Cecilia
- Garnero, Constanza
- Izaguirre, Ezequiel

Integrantes:

98919 - Cuestas, Manuel

85728 - Gallino, Gerónimo

70363 - Marchetti, Matías Hugo

94827 - Marín, Esmeralda

93368 - Ocampo, Gabriel Alejandro

94192 - Padilla, Tomás Lautaro

90313 - Picatto, Renzo Agustín

89197 - Pluda, Andrés Gonzalo

89374 - Veron Vega, Juan Sebastian

97924 - Zin, Cecilia Belen

ÍNDICE

Introducción.....	2
Guía de estilo de código.....	3
1. Convenciones de nomenclatura.....	3
Clases.....	3
Métodos y variables.....	3
Componentes (React).....	4
Hooks y funciones auxiliares.....	4
Props.....	4
2. Estilo de código.....	5
Indentación.....	5
Tamaño de archivo.....	5
Espacios en blanco.....	5
Espacios alrededor de operadores.....	5
Nombres y convenciones.....	5
3. Organización del código.....	6
Backend.....	6
Frontend.....	7
4. Pruebas.....	8
5. Justificación de decisiones de diseño.....	9
Test 1 – Inscripción válida con todos los datos correctos.....	12
Test 2 – Intento de inscripción en una actividad sin cupo disponible....	14
Test 3 – Inscripción en una actividad que no requiere vestimenta.....	16
Test 4 – Intento de inscripción fuera del horario permitido.....	18
Test 5 – Intento de inscripción sin aceptar los términos y condiciones.	20
Test 6 – Intento de inscripción sin ingresar la talla requerida.....	22
6. Decisiones de diseño de dominio.....	24

Introducción

El propósito de este documento es proporcionar un conjunto de normas para la escritura de código en nuestro equipo, con el fin de mejorar la colaboración y asegurar la calidad del desarrollo.

Mantener un estilo de codificación uniforme facilita no solo la lectura del código, sino también su mantenimiento y escalabilidad a medida que los proyectos crecen.

Estas pautas están diseñadas para garantizar que todo miembro del equipo pueda trabajar de manera coherente, siguiendo prácticas recomendadas y minimizando errores.

Guía de estilo de código

1. Convenciones de nomenclatura

Para mantener la uniformidad, legibilidad y claridad del código, se establecen las siguientes convenciones:

Clases

- Se nombran utilizando el formato **PascalCase** (por ejemplo: **InscripcionService**, **VisitanteRepository**).
- Se aplican los siguientes **sufijos** para identificar rápidamente el tipo y rol de cada clase dentro del proyecto:
 - **Service** → clases de lógica de negocio (**InscripcionService**)
 - **Repository** → acceso a datos (**VisitanteRepository**)
 - **Controller** → controladores o endpoints REST (**InscripcionController**)
 - **DTO** → objetos de transferencia de datos (**InscripcionDTO**, **VisitanteDTO**)
 - **Exception** → clases de error personalizadas (**DatosIncompletosException**)

Métodos y variables

- Utilizan el formato **camelCase** (por ejemplo: **inscribirVisitantes()**, **requiereVestimenta**).
- Deben tener nombres descriptivos y concisos que reflejen claramente su propósito.

Componentes (React)

- Se nombran utilizando el formato **PascalCase** (por ejemplo: **ActivityRegistrationForm**, **ActivitySelector**, **TimeSlotSelector**, **ParticipantForm**).
- Cada componente debe definirse en su propio archivo, cuyo nombre coincida con el del componente (por ejemplo, **ActivitySelector.jsx**).
- Los componentes deben ser pequeños, con una única responsabilidad, y evitar efectos secundarios en la función de renderizado.

Hooks y funciones auxiliares

- Utilizan formato **camelCase** (por ejemplo: **handleSubmit**, **validateParticipant**).
- Las funciones que manejan eventos deben comenzar con el prefijo **handle** (por ejemplo: **handleActivityChange**, **handleNumChange**, **handleParticipantChange**).
- Las funciones de validación o transformación de datos deben comenzar con **verbos descriptivos** (por ejemplo: **validate**, **formatData**, **calculateSlots**).

Props

- Se nombran en formato **camelCase**, y deben describir claramente su función o contenido (por ejemplo: **onChange**, **selectedActivity**, **requiresClothingSize**, **error**, **slots**).

2. Estilo de código

Mantener un estilo de código consistente ayuda a la legibilidad y a la colaboración.

Indentación

- Usar 4 espacios por nivel de indentación (no tabulaciones).
- Mantener la indentación consistente en todo el proyecto.

Tamaño de archivo

- Si un archivo crece demasiado, se deberá dividirlo en módulos o componentes para mantener la claridad y la reutilización.

Espacios en blanco

- Separar secciones lógicas del código con una línea en blanco.

Espacios alrededor de operadores

- Usar espacios alrededor de operadores (`=`, `+`, `-`, `===`, etc.) y después de comas.

Nombres y convenciones

- Usar **camelCase** para variables y funciones, y **PascalCase** para clases y componentes.
- Mantener nombres descriptivos y consistentes en todo el proyecto.

3. Organización del código

En nuestro caso, las importaciones presentan la siguiente estructura:

Backend

Las importaciones deben organizarse en el siguiente orden:

1. Importaciones internas del proyecto

Ejemplo:

```
import ar.edu.utn.frc.grupo6.tp6.tdd.ecoharmony_park_back.models.ActividadProgramada;  
import ar.edu.utn.frc.grupo6.tp6.tdd.ecoharmony_park_back.repositories.ActividadProgramadaRepository;
```

2. Importaciones externas (frameworks o librerías de terceros, como Spring)

Ejemplo:

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Service;
```

3. Importaciones de bibliotecas estándar de Java

Ejemplo:

```
import java.time.LocalDateTime;  
import java.util.List;  
import java.util.Optional;  
import java.util.stream.Collectors;
```

Frontend

Las importaciones deben organizarse en el siguiente orden:

1. **Importaciones externas** (React u otras librerías de terceros)

Ejemplo:

```
import React, { useEffect, useState } from "react";
```

2. **Importaciones internas del proyecto** (componentes, hooks, utilidades locales, etc.)

Ejemplo:

```
import ActivitySelector from "../ActivitySelector";  
import TimeSlotSelector from "../TimeSlotSelector";  
import ParticipantForm from "../ParticipantForm";
```

3. **Importaciones de estilos, imágenes o recursos**

Ejemplo:

```
import "../styles/ActivityForm.scss";
```


4. Pruebas

Se utiliza JUnit 5 (Jupiter) junto con Mockito y el entorno de pruebas de Spring Boot, lo que permite validar tanto la lógica interna de los servicios de forma aislada como la integración de los componentes dentro del contexto real de la aplicación.

Se hace uso de Mockito para las pruebas unitarias, aislando la lógica del servicio mediante las anotaciones `@ExtendWith(MockitoExtension.class)`, `@Mock` y `@InjectMocks`.

Para las pruebas de integración, se aplican las anotaciones `@SpringBootTest` para cargar el contexto de Spring y `@Transactional` para asegurar el rollback de los cambios en la base de datos después de cada prueba.

Además, se emplean aserciones de la librería `org.junit.jupiter.api.Assertions` como `assertEquals`, `assertTrue`, `assertNotNull`, entre otras, para verificar los resultados esperados.

5. Justificación de decisiones de diseño

Nuestro trabajo consta de seis tests unitarios, todos diseñados para verificar distintos comportamientos del método **inscribirVisitantes()** perteneciente al servicio de inscripciones.

Este método concentra la lógica principal del proceso de inscripción de visitantes a una actividad programada, incluyendo la validación de datos, control de cupos, verificación de términos y condiciones, y persistencia de la información en la base de datos.

Cada test se enfoca en evaluar un escenario particular, ya sea válido o inválido, con el objetivo de asegurar que el método responda correctamente ante diferentes combinaciones de datos de entrada.

De esta manera, se cubren tanto los casos exitosos (*donde la inscripción debe completarse correctamente*) como los casos de error (*donde se espera el lanzamiento de excepciones específicas ante condiciones inválidas*).

A continuación, se detallan los seis tests implementados, junto con la justificación de las decisiones de diseño tomadas y el resultado esperado en cada caso.

Método inscribirVisitantes()

```
public Inscripcion inscribirVisitantes(InscripcionDTO
inscripcionDTO) {
    // 1. Validar que se aceptaron los términos y condiciones
    if (inscripcionDTO.getAceptanTerminosYCondiciones() == null ||
        !inscripcionDTO.getAceptanTerminosYCondiciones()) {
        throw new TerminosNoAceptadosException();
    }

    // Validar que hay participantes
    if (inscripcionDTO.getParticipantes() == null ||
        inscripcionDTO.getParticipantes().isEmpty()) {
        throw new DatosIncompletosException();
    }

    // 2. Buscar la actividad programada
    ActividadProgramada actividadProgramada =
        actividadProgramadaRepository
            .findById(inscripcionDTO.getActividadProgramadaId())
            .orElseThrow(() -> new RuntimeException("Actividad
programada no encontrada"));

    // 3. Validar que hay cupo disponible para todos los
    participantes
    if (actividadProgramada.getCupoDisponible() == null ||
        actividadProgramada.getCupoDisponible() <
        inscripcionDTO.getParticipantes().size()) {
        throw new CupoInsuficienteException();
    }

    // 4. Validar que la inscripción se realiza fuera del horario
    de la actividad
    if (inscripcionDTO.getFechaHoraInscripcion() != null) {
        if
        (inscripcionDTO.getFechaHoraInscripcion().isAfter(actividadProgram
        ada.getFechaHoraInicio())) {
            throw new HorarioNoDisponibleException();
        }
    }

    // 5. Procesar y guardar los visitantes
    List<Visitante> visitantes = new ArrayList<>();
    for (VisitanteDTO visitanteDTO :
        inscripcionDTO.getParticipantes()) {
        // Validar datos básicos del visitante
        if (visitanteDTO.getNombre() == null ||
            visitanteDTO.getNombre().trim().isEmpty() ||
            visitanteDTO.getDni() == null || visitanteDTO.getEdad()
            == null) {
            throw new DatosIncompletosException();
        }

        // Validar que si la actividad requiere vestimenta, el
```

```
visitante proporcione la talla
    if
    (actividadProgramada.getActividad().getRequiereVestimenta() &&
    visitanteDTO.getTallaId() == null) {
        throw new DatosIncompletosException();
    }

    // Crear el visitante
    Talla talla = null;
    if (visitanteDTO.getTallaId() != null) {
        talla =
    tallaRepository.findById(visitanteDTO.getTallaId())
        .orElseThrow(DatosIncompletosException::new);
    }

    Visitante visitante = new Visitante(
        visitanteDTO.getNombre(),
        visitanteDTO.getDni(),
        visitanteDTO.getEdad(),
        talla
    );

    // Guardar el visitante
    Visitante visitanteGuardado =
    visitanteRepository.save(visitante);
    visitantes.add(visitanteGuardado);
}

// Actualizar el cupo disponible
actividadProgramada.setCupoDisponible(actividadProgramada.getCupoD
isponible() - visitantes.size());
    actividadProgramadaRepository.save(actividadProgramada);

// 6. Crear y guardar la inscripción
Inscripcion inscripcion = new Inscripcion(
    null, // El ID se generará automáticamente
    inscripcionDTO.getFechaHoraInscripcion(),
    actividadProgramada,
    inscripcionDTO.getAceptanTerminosYCondiciones(),
    visitantes
);

return inscripcionRepository.save(inscripcion);
}
```

Test 1 – Inscripción válida con todos los datos correctos

```
public void
testInscribirseConCupoHorarioDatosVisitanteTyCValidos() {
    // Arrange: Datos de la inscripción. La actividad con ID 3 y
    // talla con ID 1 existen en data.sql
    final Long actividadProgramadaId = 3L;
    VisitanteDTO visitanteDTO1 = new VisitanteDTO("Johnny Lojuno",
    44444444, 21, 1L);
    VisitanteDTO visitanteDTO2 = new VisitanteDTO("Johnny
    Loconozco", 44444443, 22, 1L);
    List<VisitanteDTO> participantesDTO = List.of(visitanteDTO1,
    visitanteDTO2);
    InscripcionDTO inscripcionDTO = new InscripcionDTO(
        LocalDateTime.of(2025, 10, 8, 10, 45, 0), //
        Inscripción durante el horario de la actividad (10:30-11:30)
        actividadProgramadaId,
        true,
        participantesDTO
    );

    // Act: Llamar al servicio para inscribir
    Inscripcion resultado =
    inscripcionService.inscribirVisitantes(inscripcionDTO);

    // Assert: Verificar que la inscripción se guardó correctamente
    // en la BD
    assertNotNull(resultado, "El Service debe devolver una Entidad
    de Inscripción.");
    assertNotNull(resultado.getId(), "La Inscripción debe tener un
    ID asignado por la BD.");

    // Buscar la inscripción en la BD para verificar
    Optional<Inscripcion> inscripcionGuardadaOpt =
    inscripcionRepository.findById(resultado.getId());
    assertTrue(inscripcionGuardadaOpt.isPresent(), "La inscripción
    debe existir en la base de datos.");
    Inscripcion inscripcionGuardada = inscripcionGuardadaOpt.get();

    assertTrue(inscripcionGuardada.getAceptanTerminosYCondiciones(),
    "Debe reflejar que los T&C fueron aceptados.");
    assertEquals(actividadProgramadaId,
    inscripcionGuardada.getActividadSeleccionada().getId(), "La
    Inscripción debe estar ligada a la Actividad Programada
    correcta.");
    assertEquals(2, inscripcionGuardada.getParticipantes().size(),
    "Deben haberse guardado 2 participantes.");

    // Verificar que los visitantes se guardaron
    Optional<Visitante> visitante1 =
    visitanteRepository.findByDni(44444444);
    Optional<Visitante> visitante2 =
    visitanteRepository.findByDni(44444443);
```

```
    assertTrue(visitor1.isPresent(), "El primer visitante debe  
    estar en la BD.");  
    assertTrue(visitor2.isPresent(), "El segundo visitante debe  
    estar en la BD.");  
    assertEquals("Johnny Lojuno", visitor1.get().getNombre());  
    assertEquals("Johnny Loconozco", visitor2.get().getNombre());  
}
```

Este test verifica el comportamiento esperado del método **inscribirVisitantes()** cuando se proporciona un conjunto de datos completamente válidos. Se simula una inscripción para una actividad existente, con cupo suficiente, participantes con datos completos, aceptación de términos y condiciones, y una talla válida.

Se decidió implementar este test para validar el caso de éxito principal del servicio, confirmando que el flujo completo de inscripción se ejecuta correctamente cuando no existen errores en los datos de entrada.

Además, permite comprobar la integración entre las entidades involucradas (Inscripción, ActividadProgramada y Visitante), y garantizar que las relaciones persisten adecuadamente en la base de datos.

El test también asegura que se respeten los requisitos funcionales básicos:

- La inscripción debe generarse con un identificador válido.
- Debe asociarse correctamente a la actividad programada correspondiente.
- Los visitantes deben almacenarse y vincularse correctamente a la inscripción.
- Debe reflejarse la aceptación de términos y condiciones.

Resultado esperado:

- El método debe retornar un objeto Inscripción persistido correctamente en la base de datos, con todos sus campos completos y dos visitantes asociados.
- No se deben lanzar excepciones, y las verificaciones realizadas mediante **assertNotNull**, **assertTrue** y **assertEquals** deben cumplirse exitosamente.

Test 2 – Intento de inscripción en una actividad sin cupo disponible

```
@Test
public void testInscribirseActividadSinCupo() {
    // Arrange: La actividad con ID 4 en data.sql tiene cupo 0
    final Long actividadProgramadaId = 4L;
    VisitanteDTO visitanteDTO1 = new VisitanteDTO("Esteban Quito",
    11223344, 21, 1L);
    IncripcionDTO inscripcionDTO = new IncripcionDTO(
        LocalDateTime.of(2025, 10, 7, 10, 30),
        actividadProgramadaId,
        true,
        List.of(visitanteDTO1)
    );

    long countInscripcionesAntes = inscripcionRepository.count();
    long countVisitantesAntes = visitanteRepository.count();

    // Act & Assert: Esperar la excepción y verificar que no se
    guardó nada
    assertThrows(CupoInsuficienteException.class, () ->
    inscripcionService.inscribirVisitantes(inscripcionDTO), "Debe
    fallar porque el cupo es cero.");

    assertEquals(countInscripcionesAntes,
    inscripcionRepository.count(), "No debe guardarse ninguna
    inscripción.");
    assertEquals(countVisitantesAntes, visitanteRepository.count(),
    "No debe guardarse ningún visitante.");
}
```

Este test verifica que el método **inscribirVisitantes()** responda correctamente cuando se intenta realizar una inscripción en una actividad cuyo cupo disponible es cero.

El caso utiliza una actividad existente en la base de datos (ID 4), configurada sin cupos, y un participante con datos válidos, simulando así un escenario en el que el sistema debería rechazar la solicitud.

Se decidió incluir este test para garantizar la correcta gestión de la capacidad de las actividades, evitando inscripciones por encima del cupo permitido.

Esta validación resulta esencial para mantener la consistencia del dominio y evitar la sobreasignación de recursos o plazas.

Asimismo, el test verifica que, ante la excepción esperada (CupoInsuficienteException), no se persistan datos inconsistentes en la base de datos, confirmando que el sistema mantiene su estado anterior sin registrar visitantes ni inscripciones inválidas.

Resultado esperado:

- El método debe lanzar una excepción del tipo **CupoInsuficienteException**, impidiendo la creación de la inscripción.
- Los contadores de inscripciones y visitantes en la base de datos deben permanecer sin cambios, garantizando que no se haya persistido información errónea.

Test 3 – Inscripción en una actividad que no requiere vestimenta

```
@Test
public void testInscribirseActividadNoRequiereTalle() {
    // Arrange: La actividad con ID 6 en data.sql no requiere
    // vestimenta
    final Long actividadProgramadaId = 6L;
    VisitanteDTO visitanteDTO = new VisitanteDTO("Juan Perez",
    55555555, 21, null);
    InscripcionDTO inscripcionDTO = new InscripcionDTO(
        LocalDateTime.of(2025, 10, 30, 9, 30, 0), //
        // Inscripción durante el horario de la actividad (09:00-10:30)
        actividadProgramadaId,
        true,
        List.of(visitanteDTO)
    );

    // Act: La inscripción debe ser exitosa
    Inscripcion resultado =
    inscripcionService.inscribirVisitantes(inscripcionDTO);

    // Assert: Verificar que se guardó correctamente
    assertNotNull(resultado);
    assertNotNull(resultado.getId());

    Optional<Visitante> visitanteGuardado =
    visitanteRepository.findByDni(55555555);
    assertTrue(visitanteGuardado.isPresent());
    assertNull(visitanteGuardado.get().getTallaVestimenta(), "La
    talla del visitante debe ser null.");
}
```

Este test evalúa el comportamiento del método **inscribirVisitantes()** cuando la actividad seleccionada no requiere vestimenta especial, y por lo tanto, el visitante no proporciona una talla.

Se utiliza una actividad con ID 6 (según los datos de prueba), la cual está configurada para no requerir vestimenta. El visitante posee todos los demás datos válidos y la inscripción se realiza en un horario permitido.

Se decidió implementar este test para confirmar que el sistema no exija información innecesaria en contextos donde no aplica, manteniendo así la flexibilidad del modelo de datos.

Permite comprobar que la validación de la talla se ejecuta de forma condicional: sólo cuando la actividad efectivamente requiere vestimenta.

De esta forma, se garantiza que el método maneje correctamente ambos escenarios (actividades con y sin requerimiento de vestimenta), asegurando un comportamiento coherente con las reglas del negocio.

Resultado esperado:

- La inscripción debe completarse correctamente y persistir en la base de datos.
- El visitante asociado debe almacenarse sin errores y su campo **tallaVestimenta** debe ser **null**, evidenciando que el sistema no obliga a ingresar una talla cuando la actividad no lo requiere.
- No se deben lanzar excepciones.

Test 4 – Intento de inscripción fuera del horario permitido

```
@Test
public void testInscribirseActividadHorarioNoDisponible() {
    // Arrange: La actividad con ID 3 es de 10:30 a 11:30. La
    inscripción es a las 9:00.
    final Long actividadProgramadaId = 3L;
    final LocalDateTime horaInscripcionTemprana =
    LocalDateTime.of(2025, 10, 8, 15, 0, 0);
    VisitanteDTO visitanteDTO = new VisitanteDTO("Ana Temporal",
    66666666, 21, 1L);
    InscripcionDTO inscripcionDTO = new InscripcionDTO(
        horaInscripcionTemprana,
        actividadProgramadaId,
        true,
        List.of(visitanteDTO)
    );

    long countInscripcionesAntes = inscripcionRepository.count();

    // Act & Assert
    assertThrows(HorarioNoDisponibleException.class, () ->
    inscripcionService.inscribirVisitantes(inscripcionDTO), "Debe fallar
    porque la hora de inscripción está fuera del horario.");
    assertEquals(countInscripcionesAntes,
    inscripcionRepository.count(), "No debe guardarse ninguna
    inscripción.");
}
```

Este test valida que el método **inscribirVisitantes()** arroja una excepción cuando se intenta realizar una inscripción en un horario que no está disponible para la actividad seleccionada.

En este caso, la actividad con ID 3 está programada de 10:30 a 11:30, pero la inscripción se realiza a las 15:00 horas, es decir, posterior al inicio de la actividad.

Se decidió implementar este test para garantizar que el sistema respete las restricciones temporales de las actividades programadas, evitando inscripciones fuera del horario válido.

Este control es fundamental para asegurar la coherencia temporal del proceso y mantener la integridad de las inscripciones registradas.

Además, se busca confirmar que, ante un horario no permitido, el sistema no persista ningún cambio en la base de datos ni genere entidades inválidas.

Resultado esperado:

- El método debe lanzar una excepción del tipo **HorarioNoDisponibleException**, impidiendo la creación de la inscripción.
- El conteo de inscripciones en la base de datos debe permanecer inalterado, confirmando que no se registró ninguna inscripción fuera del horario permitido.

Test 5 – Intento de inscripción sin aceptar los términos y condiciones

```
@Test
public void testInscribirseSinAceptarTerminosYCondiciones() {
    // Arrange: La inscripción tiene aceptanTerminosYCondiciones
    = false
    final Long actividadProgramadaId = 3L;
    VisitanteDTO visitanteDTO = new VisitanteDTO("Juana Pereza",
    77777777, 21, 1L);
    IncripcionDTO inscripcionDTO = new IncripcionDTO(
        LocalDateTime.of(2025, 10, 8, 10, 45, 0), //
    Inscripción durante el horario válido
        actividadProgramadaId,
        false, // No acepta T&C
        List.of(visitanteDTO)
    );

    long countInscripcionesAntes = inscripcionRepository.count();

    // Act & Assert
    assertThrows(TerminosNoAceptadosException.class, () ->
    inscripcionService.inscribirVisitantes(inscripcionDTO), "Debe fallar
    porque los T&C no fueron aceptados.");
    assertEquals(countInscripcionesAntes,
    inscripcionRepository.count(), "No debe guardarse ninguna
    inscripción.");
}
```

Este test verifica que el método **inscribirVisitantes()** rechace la inscripción cuando el usuario no acepta los términos y condiciones.

Se simula una inscripción válida en cuanto a horario, actividad y datos del visitante, pero con el campo **aceptanTerminosYCondiciones** establecido en **false**.

Se decidió incluir este test para garantizar el cumplimiento de una regla de negocio fundamental: la aceptación de los términos y condiciones es un requisito obligatorio antes de registrar cualquier inscripción.

Este control protege tanto la integridad del sistema como los aspectos legales asociados al consentimiento del participante.

Además, el test permite verificar que, al producirse la excepción esperada (**TerminosNoAceptadosException**), no se persisten inscripciones inválidas en la base de datos.

Resultado esperado:

- El método debe lanzar una excepción del tipo **TerminosNoAceptadosException**, impidiendo el registro de la inscripción.
- El conteo de inscripciones debe mantenerse sin cambios, confirmando que el sistema no guarda información cuando no se cumplen las condiciones requeridas.

Test 6 – Intento de inscripción sin ingresar la talla requerida

```
@Test
public void testInscribirseActividadTalleRequeridoNoIngresado() {
    // Arrange: La actividad con ID 3 requiere vestimenta, pero
    // el visitante no provee talla.
    final Long actividadProgramadaId = 3L;
    VisitanteDTO visitanteDTO = new VisitanteDTO("Dani
    Vestimenta", 88888888, 21, null); // Talla es null
    IncripcionDTO inscripcionDTO = new IncripcionDTO(
        LocalDateTime.of(2025, 10, 8, 10, 0, 0), //
        Incripción durante el horario válido
        actividadProgramadaId,
        true,
        List.of(visitanteDTO)
    );

    long countInscripcionesAntes = inscripcionRepository.count();

    // Act & Assert
    assertThrows(DatosIncompletosException.class, () ->
        inscripcionService.inscribirVisitantes(inscripcionDTO), "Debe fallar
        porque la talla es requerida y no se proveyó.");
    assertEquals(countInscripcionesAntes,
        inscripcionRepository.count(), "No debe guardarse ninguna
        inscripción.");
}
```

Este test evalúa el comportamiento del método **inscribirVisitantes()** cuando la actividad seleccionada requiere vestimenta y el visitante no proporciona una talla.

Se utiliza una actividad existente (ID 3) configurada para requerir vestimenta, y un visitante con datos válidos pero con el campo **tallald** en **null**.

Se decidió incluir este test para validar que el sistema exija información obligatoria según las características de la actividad.

El control de la talla en actividades que requieren vestimenta asegura la completitud y coherencia de los datos registrados, evitando inconsistencias que podrían afectar la preparación de materiales o la gestión logística.

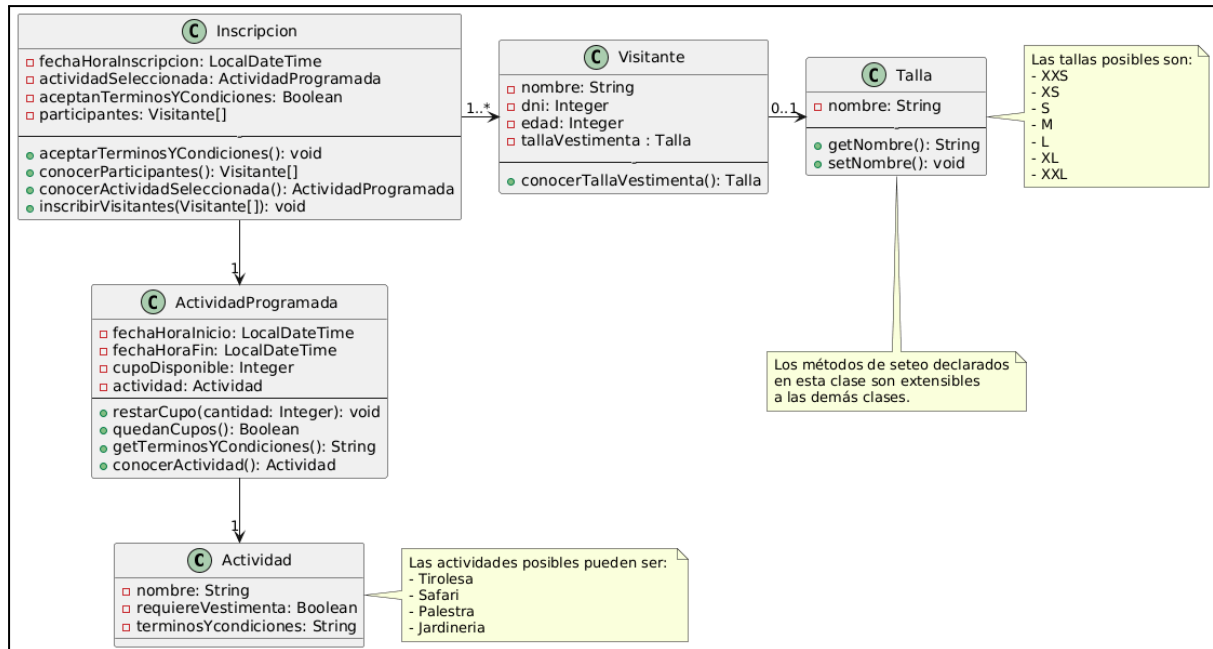
Además, el test confirma que, ante una omisión de datos obligatorios, se lance la excepción correspondiente (**DatosIncompletosException**) y que no se persistan cambios en la base de datos.

Resultado esperado:

- El método debe lanzar una excepción del tipo **DatosIncompletosException**, impidiendo la creación de la inscripción.
- El número de inscripciones en la base de datos debe permanecer inalterado, garantizando que no se haya guardado información incompleta o inválida.

6. Decisiones de diseño de dominio

Para realizar la correspondiente implementación y cumplir con el ciclo Diseñar > Probar > Desarrollar, construimos el siguiente dominio como referencia general:



Tomando como referencia el diagrama UML planteado anteriormente, se plantea a posterior un diagrama Entidad-Relación que facilita la referencia en el momento de la implementación de persistencia.

